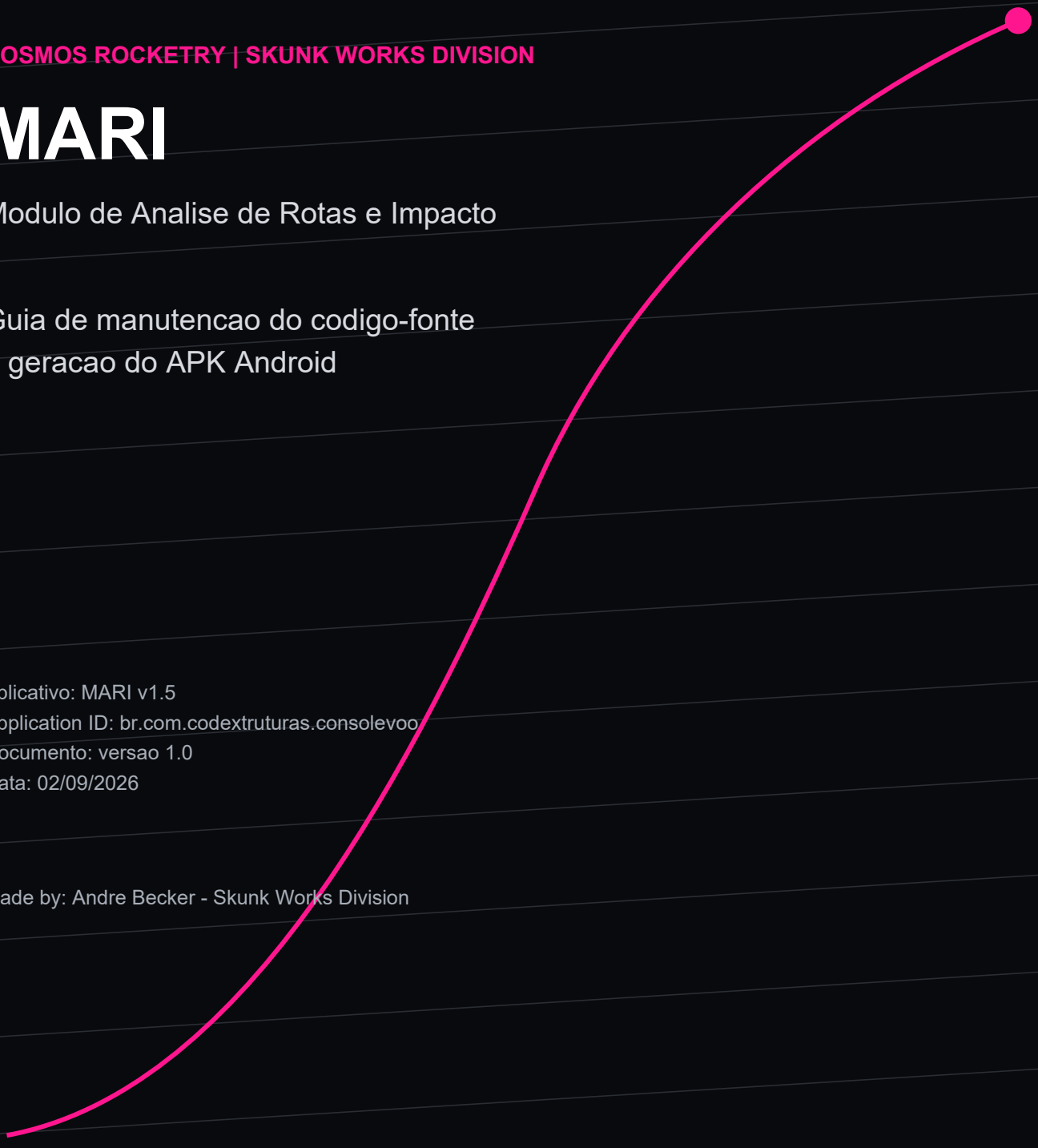
KOSMOS ROCKETRY | SKUNK WORKS DIVISION

MARI

Modulo de Analise de Rotas e Impacto

Guia de manutencao do codigo-fonte
e geracao do APK Android



Aplicativo: MARI v1.5
Application ID: br.com.codextruturas.consolevoo
Documento: versao 1.0
Data: 02/09/2026

Made by: Andre Becker - Skunk Works Division

1. Objetivo e escopo desta entrega

Este manual permite que a equipe da Kosmos Rocketry mantenha o MARI, altere o codigo e gere novos APKs localmente, sem publicar o projeto na internet. O simulador, o Pyodide, o RocketPy e os recursos necessarios para o modo offline sao incorporados ao APK durante a compilacao.

> **Limite de responsabilidade tecnica**
>
> Alteracoes em massa, centro de gravidade, inercia, arrasto, empuxo, paraquedas, geometria ou atmosfera mudam o modelo fisico. Esses valores devem ser aprovados pela equipe responsavel antes de uma versao ser usada em operacao de lancamento.

A entrega e formada por quatro itens:

- Codigo-fonte limpo do MARI, sem pastas de cache, sem o site hospedado e sem credenciais online.
- APK release v1.5 assinado com a nova chave oficial da Kosmos Rocketry.
- Kit de assinatura com a chave privada e o certificado publico.
- Arquivo de credenciais separado, contendo as senhas da assinatura.

A publicacao do site nao faz parte deste fluxo. Alterar e gerar o APK nao exige conta ou permissao de hospedagem.

Conteudo recomendado para envio

Arquivo	Enviar para	Observacao
MARI-Kosmos-codigo-fonte-v1.5.zip	Equipe de desenvolvimento	Codigo, manual e script de compilacao.
MARI-v1.5-KOSMOS-release.apk	Usuarios e testadores	Primeira versao com a assinatura oficial.
MARI-Kosmos-chave-assinatura.zip	Responsavel por releases	Enviar por canal privado.
CREDENCIAIS-ASSINATURA-KOSMOS-MARI.txt	Responsavel por releases	Enviar por outro canal, separado da chave.

2. Como o aplicativo esta organizado

O MARI usa uma interface web instalada dentro de um aplicativo Android. O Capacitor coloca essa interface em uma WebView. O Pyodide executa Python em WebAssembly dentro dessa WebView, e o RocketPy calcula a simulacao. O Cesium mostra a trajetoria e os modelos 3D.

Camada	Tecnologia	Funcao
Interface	React + TypeScript	Formularios, validacao, resultados, graficos e navegacao.
Aplicativo Android	Capacitor + WebView	Empacota a interface como APK e fornece integracao com Android.
Python no celular	Pyodide + WebAssembly	Executa Python sem instalar Python no Android.
Simulacao	RocketPy	Monta ambiente, motor, foguete, voo e paraquedas.
Mapa e animacao	Cesium + modelos GLB	Exibe terreno, trajetoria, atitude e recuperacao 3D.

Fluxo dos dados

```
Campos preenchidos na tela
        -> app/page.tsx valida e monta os parametros
        -> public/simulation/worker.js envia ao Pyodide
        -> public/simulation/deimos_core.py executa o RocketPy
        -> resultados retornam para a interface
        -> graficos, mapa 3D, KML e animacao sao apresentados
```

Arquivos principais

Caminho relativo	O que alterar
app/page.tsx	Tela, textos, campos, valores padrao, validacao e exibicao dos resultados.
public/simulation/deimos_core.py	Fisica da simulacao, motores, foguetes, paraquedas e saidas.
public/simulation/worker.js	Carga do Pyodide, pacotes Python e comunicacao com a interface.
public/trajectory-map.html	Mapa, cameras, animacao 3D, modelos, coifa e paraquedas.
public/models/	Modelos tridimensionais GLB do Atlas e do DEIMOS II.
public/simulation/assets/	Curvas de empuxo, arrasto e demais arquivos da simulacao.
capacitor.config.ts	Nome, identificador e pasta da versao Android.
android/app/build.gradle	Versao, assinatura e configuracoes Android.

3. Requisitos do computador

Item	Requisito recomendado	Como conferir
Sistema	Windows 10 ou 11, 64 bits	Configuracoes > Sistema > Sobre
Node.js	22.13 ou superior	node --version
pnpm	Versao atual compativel	pnpm --version
Android Studio	Versao atual estavel	Help > About
Android SDK	Platform 36, Build Tools e Platform Tools	SDK Manager
Java	JDK 21	java -version
Editor	VS Code ou Android Studio	Opcional para edicao
Espaco livre	Pelo menos 12 GB	Necessario para SDK, Gradle e dependencias
Internet	Obrigatoria na primeira preparacao	Depois, builds podem usar o cache local

Instalacao do Node.js e pnpm

- Instale o Node.js 22 LTS ou uma versao superior compativel.
- Abra um PowerShell novo e confira node --version.
- Habilite o gerenciador pnpm com Corepack.

```
node --version
corepack enable
corepack prepare pnpm@latest --activate
pnpm --version
```

Instalacao do Android

- Instale o Android Studio e abra SDK Manager.
- Em SDK Platforms, instale Android API 36.
- Em SDK Tools, instale Android SDK Build-Tools, Platform-Tools e Command-line Tools.
- Use o JDK 21 incluido no Android Studio ou instale um JDK 21 separado.

> **Caminho recomendado**
>
> Extraia o projeto em um caminho curto, por exemplo C:\Projetos\MARI. Caminhos muito longos ou sincronizados pelo OneDrive podem aumentar o risco de erros do Gradle.

4. Preparacao inicial do projeto

Esta etapa e feita uma vez em cada computador de desenvolvimento.

1. Extraia o codigo

- Extraia MARI-Kosmos-codigo-fonte-v1.5.zip em C:\Projetos\MARI.
- Nao trabalhe diretamente dentro do ZIP.

2. Instale as dependencias

```
cd C:\Projetos\MARI
pnpm install --frozen-lockfile
```

3. Configure o Android SDK

Abra a pasta android no Android Studio. O programa normalmente cria android/local.properties. Se precisar criar manualmente, use o caminho real do SDK do computador:

```
sdk.dir=C:\\Users\\SEU_USUARIO\\AppData\\Local\\Android\\Sdk
```

4. Instale a chave oficial

- Extraia MARI-Kosmos-chave-assinatura.zip em um local protegido.
- Copie kosmos-mari-release.jks para android\signing\kosmos-mari-release.jks.
- Receba as senhas por um canal separado e nao as coloque no Git.

5. Teste a interface localmente

```
pnpm dev
```

O terminal exibira um endereco local. Esse teste e util para conferir formularios e resultados, mas o teste final deve ser feito no APK instalado em um aparelho Android.

> **Nao use o APK debug como versao oficial**
>
> O comando pnpm android:apk gera uma versao de desenvolvimento. Para distribuir atualizacoes oficiais, use sempre scripts\build-release-apk.ps1 com a chave da Kosmos.

5. Exemplo 1: alterar um texto da interface

Objetivo: trocar a descricao do motor exibida no cartao de configuracao do DEIMOS II.

Arquivo: app/page.tsx

Localize o trecho:

```
{vehicle === 'atlas'
  ? 'Atlas com motor Proton.'
  : 'DEIMOS II com motor FAÍSCA POWER.'}
```

Altere apenas o texto desejado:

```
{vehicle === 'atlas'
  ? 'Atlas com motor Proton.'
  : 'DEIMOS II com motor FAÍSCA POWER V2.'}
```

Depois, salve o arquivo e execute pnpm dev. Abra DEIMOS II e confira a descricao. Esse tipo de alteracao nao muda a simulacao, apenas o texto exibido.

O que evitar

- Nao remova chaves, parenteses ou aspas do JSX.
- Nao altere o texto do Atlas por engano dentro do operador vehicle === 'atlas'.
- Se a tela ficar em branco, leia o primeiro erro exibido no terminal.

6. Exemplo 2: alterar valores padrao de entrada

Objetivo: mudar os valores que aparecem automaticamente quando o usuario seleciona o DEIMOS II. Isso altera apenas o preenchimento inicial. O usuario ainda pode editar os campos antes de simular.

Arquivo: app/page.tsx

Trecho atual do objeto DEIMOS_FORM:

```
const DEIMOS_FORM: FormState = {
  rocket_mass_kg: '2.790',
  cg_from_tail_m: '0.686',
  number_of_simulations: '1',
  inclination_deg: '80',
  rail_length_m: '4.0',
  heading_deg: '330',
  // outros campos...
};
```

Exemplo: nova massa padrao de 2,850 kg e inclinacao de 82 graus:

```
const DEIMOS_FORM: FormState = {
  rocket_mass_kg: '2.850',
  cg_from_tail_m: '0.686',
  number_of_simulations: '1',
  inclination_deg: '82',
  rail_length_m: '4.0',
  heading_deg: '330',
  // outros campos...
};
```

> **Convencoes importantes**
>
> A massa informada e a massa do foguete sem motor. No DEIMOS II, o CG e medido desde a cauda. No Atlas, o campo cg_reference_m e medido desde a ponta da coifa. Use ponto como separador decimal dentro do codigo.

Se alterar limites operacionais, revise tambem a funcao validate() no mesmo arquivo. Nao aumente os limites apenas para fazer um valor incorreto passar pela validacao.

7. Exemplo 3: alterar um parametro fisico do RocketPy

Objetivo: alterar o produto CdS nominal do paraquedas principal do DEIMOS II. Essa e uma mudanca fisica e deve ser aprovada pela equipe de aerodinamica.

Arquivo: public/simulation/deimos_core.py

Trecho atual dentro de build_deimos_rocket:

```
mass = float(params["rocket_mass_kg"])
cg = float(params["cg_from_tail_m"])
thrust_scale = 1.0
parachute_cd_s = 1.561
parachute_lag = 5.5 if params["flight_case"] == "nominal" else 3.5
```

Exemplo: valor novo de CdS igual a 1,620 m2:

```
parachute_cd_s = 1.620
```

O mesmo valor e usado na criacao do paraquedas:

```
rocket.add_parachute(
    name="Main",
    cd_s=parachute_cd_s,
    trigger=trigger,
    lag=parachute_lag,
)
```

Depois da mudanca, compare com uma versao de referencia: apogeu, velocidade maxima, tempo de voo, velocidade de impacto, deriva e momento de acionamento. Registre a origem do novo valor e quem o aprovou.

> **CdS nao e apenas o diametro**
>
> CdS e o produto do coeficiente de arrasto pela area de referencia. Mudar apenas o diametro visual do paraquedas no mapa nao altera a fisica. Mudar apenas CdS nao atualiza automaticamente o tamanho visual.

8. Exemplo 4: alterar a animacao 3D

Objetivo: mudar o diametro visual do paraquedas do DEIMOS II de 1,40 m para 1,50 m. Essa alteracao afeta somente o desenho 3D.

Arquivo: public/trajectory-map.html

```
deimos: {
  name: 'DEIMOS II',
  bodyModel: '/models/deimos-ii-body.glb',
  noseModel: '/models/deimos-ii-nose.glb',
  defaultParachuteDiameter: 1.4,
  closeCameraMinimum: 2.4,
  closeCameraMaximum: 14,
  closeCameraRange: 10.2,
},
```

Altere para:

```
defaultParachuteDiameter: 1.5,
```

Para manter os metadados visuais coerentes, confira tambem _parachute_visual no arquivo public/simulation/deimos_core.py:

```
return {"kind": "main", "diameter_m": 1.50}
```

Teste a abertura, a corda, a coifa, a inversao do foguete, o balanco durante a descida, o play, a pausa, o controle de tempo e as duas cameras.

Outras alteracoes 3D comuns

- Trocar modelos: substitua os arquivos GLB em public/models e mantenha os nomes esperados.
- Ajustar camera proxima: altere closeCameraMinimum, closeCameraMaximum e closeCameraRange.
- Ajustar balanco: revise RECOVERY_SWAY_PITCH_RAD e RECOVERY_SWAY_ROLL_RAD.

9. Versao do aplicativo e compatibilidade de atualizacao

Antes de gerar cada APK novo, abra android/app/build.gradle e aumente a versao.

Versao atual:

```
versionCode 6
versionName "1.5"
```

Exemplo para a proxima entrega:

```
versionCode 7
versionName "1.6"
```

Campo	Regra
versionCode	Numero inteiro. Deve sempre aumentar. O Android usa esse numero para decidir se e uma atualizacao.
versionName	Texto apresentado ao usuario. Recomenda-se 1.6, 1.7, 2.0 e assim por diante.
applicationId	Deve permanecer br.com.codextruturas.consolevoo para atualizar o mesmo aplicativo.
Assinatura	Deve usar sempre kosmos-mari-release.jks e o alias kosmos-mari-release.

Reinstalacao unica obrigatoria

O APK anterior foi assinado com uma chave de desenvolvimento. O APK oficial da Kosmos usa uma nova identidade. Portanto, sera necessario desinstalar a versao anterior uma vez e instalar MARI-v1.5-KOSMOS-release.apk. Depois disso, as proximas versoes oficiais poderao ser instaladas por cima, desde que usem a mesma chave e um versionCode maior.

10. Geracao do APK oficial

Use o script incluido no projeto. Ele solicita as senhas sem grava-las no codigo, prepara os arquivos web, sincroniza o Capacitor, executa o Gradle e copia o APK final para output.

No PowerShell, dentro da raiz do projeto:

```
powershell -ExecutionPolicy Bypass `
  -File .\scripts\build-release-apk.ps1
```

O script pedira:

- Senha do arquivo de assinatura.
- Senha da chave kosmos-mari-release.

Saida esperada:

```
output\MARI-Kosmos-release.apk
```

Para pular a reinstalacao das dependencias quando pnpm install ja foi executado:

```
powershell -ExecutionPolicy Bypass `
  -File .\scripts\build-release-apk.ps1 `
  -SkipInstall
```

O que acontece internamente

- pnpm install restaura as dependencias exatas do pnpm-lock.yaml.
- build:native gera a interface Android em native-dist.
- Capacitor sync copia a interface e os recursos para o projeto Android.
- Gradle assembleRelease compila e assina o APK.
- O script copia o resultado para a pasta output.

> **Nunca compartilhe a senha no codigo**
>
> Nao escreva senhas em package.json, build.gradle, scripts, commits ou mensagens publicas. O arquivo android/keystore.properties e ignorado pelo Git, mas o metodo recomendado continua sendo o script com solicitacao interativa.

11. Plano minimo de testes antes de distribuir

Cada release deve ser testado no Galaxy S24 Ultra e, quando disponivel, no tablet Galaxy Tab S10 Lite ou no aparelho final equivalente.

Teste	Criterio de aprovacao
Instalacao limpa	APK instala, mostra nome e icone corretos e abre sem tela branca.
Atualizacao	Uma versao com versionCode maior instala sobre a versao oficial anterior.
DEIMOS II padrao	Simulacao conclui e resultados principais permanecem coerentes.
Atlas padrao	Simulacao conclui e resultados principais permanecem coerentes.
Graficos	Opcoes Nao e Sim funcionam; todas as figuras esperadas aparecem.
Atmosfera padrao	Funciona em modo aviao, sem rede.
Atmosfera Windy	Com internet, carrega o perfil; sem internet, informa a indisponibilidade.
Mapa 3D	Trajetoria, cameras, zoom, play, pausa e controle deslizante funcionam.
Atitude	Ponta da coifa acompanha a tangente da trajetoria durante a subida.
Recuperacao	Coifa, corda e paraquedas acompanham o conjunto e se orientam corretamente.
KML	Arquivo e gerado e pode ser aberto em um visualizador compativel.
Entradas limite	Validacoes impedem valores impossiveis e aceitam valores operacionais.

Registre para cada teste: versao, aparelho, versao do Android, data, responsavel, resultado e observacoes. Nao distribua uma versao cuja simulacao nominal tenha mudado sem explicacao.

12. Problemas comuns

Sintoma	Causa provavel	Correcao
pnpm nao e reconhecido	pnpm nao foi habilitado	Execute corepack enable e abra outro PowerShell.
SDK location not found	local.properties ausente ou errado	Abra android no Android Studio ou corrija sdk.dir.
JAVA_HOME incorreto	JDK ausente ou versao errada	Configure JDK 21 no Android Studio ou no sistema.
App nao instalado	Assinatura diferente da instalada	Desinstale a versao antiga apenas na migracao para a chave oficial.
Version downgrade	versionCode nao aumentou	Aumente versionCode e gere novamente.
Tela branca	Erro no JavaScript ou recursos nao sincronizados	Leia o terminal e execute a compilacao release novamente.
Windy indisponivel	Sem internet, CORS ou servico externo	Use atmosfera padrao e verifique a rede.
Build muito lento	Primeira compilacao ou cache vazio	Aguarde downloads; as proximas compilacoes tendem a ser mais rapidas.
Modelo 3D desalinhado	Origem ou rotacao do GLB mudou	Restaure o modelo anterior e ajuste noseBase, nosePivot e rotacao.

Se o problema surgiu depois de uma alteracao, reverta somente essa alteracao e repita o teste. Evite modificar varios sistemas ao mesmo tempo sem commits separados.

13. Controle de versoes e seguranca

O melhor formato para trabalho em equipe e um repositorio Git privado pertencente a Kosmos Rocketry. O ZIP desta entrega funciona como ponto inicial e copia de seguranca.

Fluxo recomendado

- Crie uma tarefa ou descricao curta para cada mudanca.
- Crie uma branch, por exemplo feature/novo-motor-deimos.
- Altere somente os arquivos necessarios.
- Execute os testes locais e gere um APK de teste.
- Outra pessoa revisa as mudancas fisicas e operacionais.
- Aumente a versao, gere o APK oficial e registre o SHA-256.
- Crie uma tag, por exemplo mari-v1.6.

Protecao da assinatura

- Mantenha duas copias criptografadas da chave em locais fisicamente separados.
- Guarde as senhas separadamente da chave.
- Restrinja a chave aos responsaveis por releases.
- Nunca gere outra chave para substituir uma chave que ainda existe.
- Nunca envie o arquivo .jks para repositorio, nuvem publica ou grupo aberto.

Por que a chave e critica

O Android vincula cada aplicativo a sua assinatura. Sem a chave oficial, nao e possivel publicar uma atualizacao reconhecida como sendo do mesmo aplicativo. Com a chave, outra pessoa pode assinar um APK que os celulares tratarao como oficial.

14. Dados da versao oficial entregue

Campo	Valor
Nome	MARI
Significado	Modulo de Analise de Rotas e Impacto
Versao	1.5
versionCode	6
Application ID	br.com.codextruturas.consolevoo
Android minimo	API 24
Android alvo	API 36
Arquivo	MARI-v1.5-KOSMOS-release.apk
SHA-256 do APK	DF1A4AFBCFE1766595BA2CDDD3FFC6603A8D81BA5B54E9EC0ED45649583B3C07
Alias da assinatura	kosmos-mari-release
Certificado	CN=MARI Android Release, OU=Skunk Works Division, O=Kosmos Rocketry, C=BR
SHA-256 do certificado	FD:6D:F0:0B:B3:18:A3:D9:DF:E0:45:87:C9:A0:BB:18:8E:FF:60:68:92:B7:36:03:50:C1:CE:47:2E:3D:1A:8E
Chave	RSA 4096 bits, SHA256withRSA, validade de 10000 dias

> **Checklist de recebimento**
>
> Confirme que o ZIP do codigo abre, que o PDF esta dentro da pasta docs, que o APK possui o SHA-256 informado, que a chave esta no pacote separado e que as credenciais foram recebidas por outro canal.

Fim do documento

Kosmos Rocketry - Skunk Works Division
MARI - Modulo de Analise de Rotas e Impacto